# TDC-CiM: Time-to-Digital Converter-Based Resonant Compute-in-Memory for INT8 CNNs With Layer-Optimized SRAM Mapping

Dhandeep Challagundla, *Graduate Student Member, IEEE*, Ignatius Bezzam, *Member, IEEE*, and Riadul Islam, *Senior Member, IEEE*

***Abstract*—In recent years, Compute-in-memory (CiM) architectures have emerged as a promising solution for deep neural network (NN) accelerators. Multiply-accumulate (MAC) is considered a *de* facto unit operation in NNs. By leveraging the inherent parallel processing capabilities of CiM, NNs that require numerous MAC operations can be executed more efficiently. This is further facilitated by storing the weights in SRAM, reducing the need for extensive data movement and enhancing overall computational speed and efficiency. Traditional CiM architectures execute MAC operations in the analog domain, employing an Analog-to-Digital converter (ADC) to convert the analog MAC values into digital outputs. However, these ADCs introduce a significant increase in area and power consumption, as well as introduce non-linearities. This work proposes a resonant time-domain compute-in-memory (TDC-CiM) architecture that eliminates the need for an ADC by using a time-to-digital converter (TDC) to digitize analog MAC results with lower power and area cost. A dedicated 8T SRAM cell enables reliable bitwise MAC operations, while the readout uses a 4-bit TDC with pulse-shrinking delay elements, achieving 1 GS/s sampling with a power consumption of only 1.25 mW. In addition, a weight-stationary data mapping strategy combined with an automated SRAM macro selection algorithm enables scalable and energy-efficient deployment across CNN workloads. Evaluation across six CNN models shows that the algorithm reduces inference energy consumption by up to 8× when scaling SRAM size from 32 KB to 256 KB, while maintaining minimal accuracy loss after quantization. The feasibility of the proposed architecture is validated on an 8 KB SRAM memory array using TSMC 28 nm technology. The proposed TDC-CiM architecture demonstrates a throughput of 320 GOPS with an energy efficiency of 38.46 TOPS/W.**

***Index Terms*—Static random access memory (SRAM), compute-in-memory (CiM), convolutional neural network (CNN), multiply-accumulate (MAC), time-to-digital converter (TDC).**

## I. Introduction

BASED on the Von Neumann architecture, neural network accelerators are currently being implemented in edge devices for complex tasks. These networks require substantial memory for accessing inputs and weights, creating memory wall bottlenecks that diminish the processor's performance, as shown in Figure 1(a). Computing-in-Memory (CiM) architectures reduce the energy overhead associated with data movement by leveraging parallel computation within DRAM [1], [2], [3] and SRAM [4], [5] memory arrays. Like conventional neural networks, the multiply-accumulate (MAC) operation is a fundamental process in CiM computations. CiM architectures execute several multiplications between inputs and weights concurrently, summing the results in the current domain. The voltage of the bitline corresponds to the final output of these MAC processes, as shown in Figure 1(b). Subsequently, an analog-to-digital converter (ADC) is utilized to convert the analog voltage into digital output bits. However, this analog processing, as shown in Figure 1(c), accounts for a prohibitively large amount (i.e., up to 59%) of the total power consumption in a CiM architecture [6].

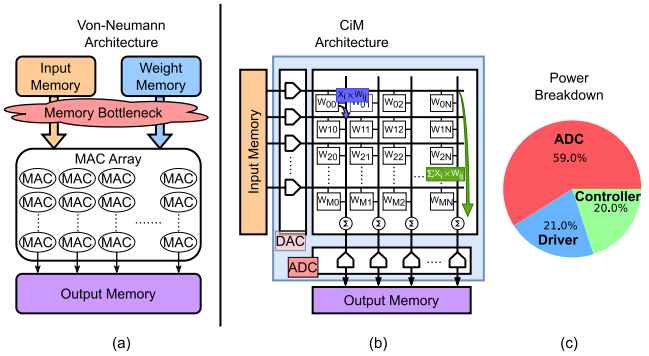


Fig. 1. (a) Traditional neural network accelerators use process elements (PEs) for MAC computation, which causes memory wall bottlenecks due to high data movement. (b) CiM architectures mitigate the need for frequent data movements by enabling analog MAC computation within memory elements using charge accumulation and ADC converters, and (c) power breakdown of conventional CiM architectures shows 59% of the total power consumption is caused by ADCs [6].

Minimizing the overhead associated with ADCs is pivotal to improving the energy efficiency of CiM accelerators. Some CiM architectures achieve this by reducing the ADC precision for sparse inputs, whereas other approaches employ reduced precision ADCs with non-linear quantization techniques [7]. A primary issue in the integration of ADCs within mixed-signal design architectures, such as CiMs, is ensuring consistent ADC performance across diverse operating conditions and scaling with technology. Additionally, the verification of analog circuits along with digital

Received 30 July 2025; revised 17 November 2025; accepted 9 December 2025. Date of publication 17 December 2025; date of current version 15 June 2026. This work was supported in part by the National Science Foundation (NSF) under Award 2138253, in part by Rezonent Inc. under Award CORP0061, and in part by the University of Maryland Baltimore County (UMBC) Startup Fund. This article was recommended by Guest Editor X. Li. *(Corresponding author: Riadul Islam.)*

Dhandeep Challagundla and Riadul Islam are with the Department of Computer Science and Electrical Engineering, University of Maryland, Baltimore County, Baltimore, MD 21250 USA (e-mail: riaduli@umbc.edu).

Ignatius Bezzam is with Rezonent Inc., Milpitas, CA 95035 USA (e-mail: i@rezonent.us).

Digital Object Identifier 10.1109/JETCAS.2025.3645585

components necessitates high-cost advanced testing methodologies due to the impact of process variations.

To overcome the non-linearity and high power consumption of ADCs, more research is focused on Time-domain ADCs (TD ADCs) in which the analog voltage is converted to delay that can be processed in the digital domain [8], [9]. In this work, we alleviate the ADC issues in CiM computation by introducing an ADC-less Resonant time-domain CiM (TDC-CiM) architecture using a time-to-digital converter (TDC) that performs the same functionality as an ADC while mimicking the behavior of a digital circuit to perform MAC operations within the SRAM memory elements [10]. In particular, the main contributions of this work are:

- First-ever ADC-less resonant CiM architecture for MAC operations utilizing a new TDC.
- A dedicated read-port 8T SRAM cell that enables read-disturb-free bitwise multiplications.
- An automated SRAM macro selection algorithm to map any given quantized CNN workload to the most energy-efficient TDC-CiM configuration by balancing kernel dimensions, required parallelism, and weight stationary mapping constraints.
- Functionality validation and robustness analysis of the TDC, demonstrating an energy efficiency of 163 fJ per conversion step.

## II. Background

As an emerging paradigm, SRAM-based CiM architectures have shown promising potential in significantly enhancing processing speed and energy efficiency for a wide range of computing tasks, such as MAC [11], [12], [13], [14], [15], CAM [16], and boolean logic [17]. Besides, both series [18], [19] and parallel resonance [20], [21] exhibit tremendous potential in energy-efficient computing. While series resonance allows a wide operating frequency range, this research deploys a parallel resonance scheme for the fixed operating frequency used in the SRAM characterization.

CiM architectures utilizing standard 6T SRAM cells [11], [13] perform computations by propagating information across the bitlines. This computational approach requires multi-row activations to fetch several operands within a single access cycle. However, this leads to high voltage changes along the bitlines, causing serious read-disturb issues potentially leading to data corruption. Reference [15] employs a 7T SRAM cell for MAC computations by leveraging the discharge of the read bitline to eliminate read-disturb issues. However, this method may lead to reverse current flow if the voltage of the read bitline has significantly dropped, while one row might not discharge, it could instead cause an increase in voltage on the bitline, inducing non-idealities in MAC computation and affecting the accuracy. Reference [14] utilizes a 9T1C architecture for MAC operations but suffers from high latency, primarily due to the time consumed during charging the capacitor during bitwise multiplication operations. Reference [16] uses an 11T SRAM cell to perform Boolean and CAM operations while eliminating the need for column-wise computation constraints. However, it suffers from high area consumption associated with the additional transistors used in the SRAM cell. This work uses an 8T SRAM bitcell that provides full read bitline swing that avoids the reverse current issue reported in [15], and reduces the area overhead compared to the 11T bitcell in [16].

CiM architectures generally employ MAC operations using two design approaches: analog-based [14] and digital-computing [22]. The first type of CiM architectures executes MAC computations using current [23], voltage [24], or pulse width [25] to represent information. These architectures benefit from low power consumption but suffer from various non-idealities. Conversely, the computation results in the digital CiM architectures are accurate when multi-bit operations are applied but suffer from high power consumption. Charge domain CiM is effective because accumulation happens on capacitors and charge adds linearly with low loss, where multiple rows can be activated while maintaining accuracy [26], [27]. Using metal oxide metal capacitors that are stable across process, voltage, and temperature (PVT), these arrays support higher parallelism per operation than current domain designs and consume lower energy [28], [29]. The ADC at readout has been the limiting element in many prior systems. Increasing the resolution of ADC to obtain higher precision operation lowers quantization error but also narrows the 1 LSB voltage step. Hence the same noise occupies a larger fraction of a code, and ADC errors become more frequent. The analog front end of the ADC also dominates power and is hard to guard across corners, which increases the energy consumption and introduces partial sum errors that reduce the overall accuracy [30]. To address these issues, this work introduces an ADC-less CiM architecture that performs the MAC computation and converts the analog MAC value into a digital output through digital TDC circuitry. The TDC-CiM uses low-energy computation using capacitor-based computing while avoiding the ADC cost by mapping the MAC voltage swing to time and digitizing with a compact TDC built from digital blocks. The TDC is co-designed with the MAC discharge range and provides a monotonic 4-bit readout that can be linearized with a small lookup table if needed, preserves INT8 inference accuracy, and lowers converter power by removing the high-resolution ADC from the loop.

Numerous studies on TDC circuits have been documented in the literature. The TDC architecture in [31] employs a delay element incorporated with a delay-locked loop and a counter to count the number of pulses resulting in a digital output. Another approach [32] utilizes two pulse-shrinking delay lines and a delay stabilization loop to convert a pulse input into digital output. Reference [9] uses a 2-step TDC and a voltage-to-time converter to produce a digital output for a given input. Unlike the previous TDC works, this work uses a simple pulse-shrinking delay element alongside DFFs to capture the pulse count depending on the voltage input, further encoded as digital bits.

## III. Proposed Architecture

In a convolutional neural network (CNN), the multiply-accumulate (MAC) operation is a fundamental computation.

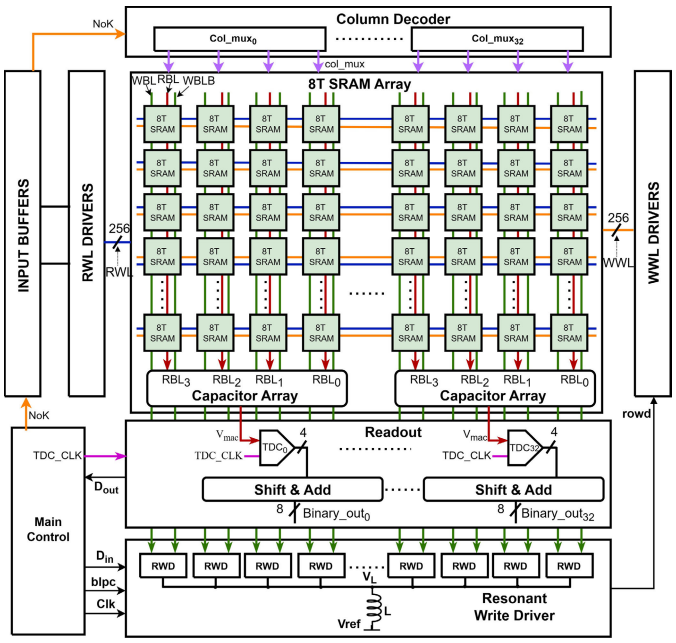


Fig. 2. The proposed architecture comprises the 8T bitcell array, RWL drivers to propagate the inputs, a new TDC-based readout circuit for executing MAC computations, and a resonant write driver for the writeback of MAC outputs.

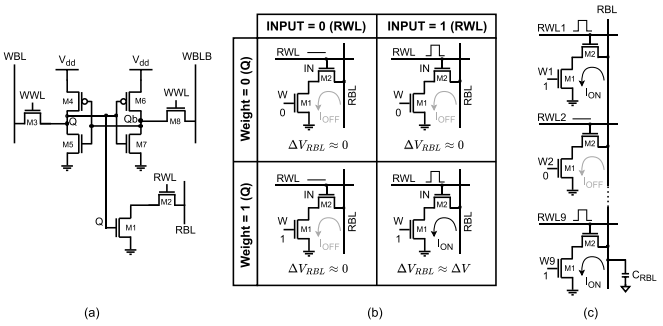


Fig. 3. (a) Schematic of 8T SRAM bitcell with $M1-M2$ transistors forming a dedicated read port, (b) bitwise multiplication of inputs and weights using 8T bitcells show discharge of $\Delta V$ only when Input and Weight are "1, " (c) multirow activation executes series of bitwise multiplications that collectively perform a bitwise multiply-accumulate operation.

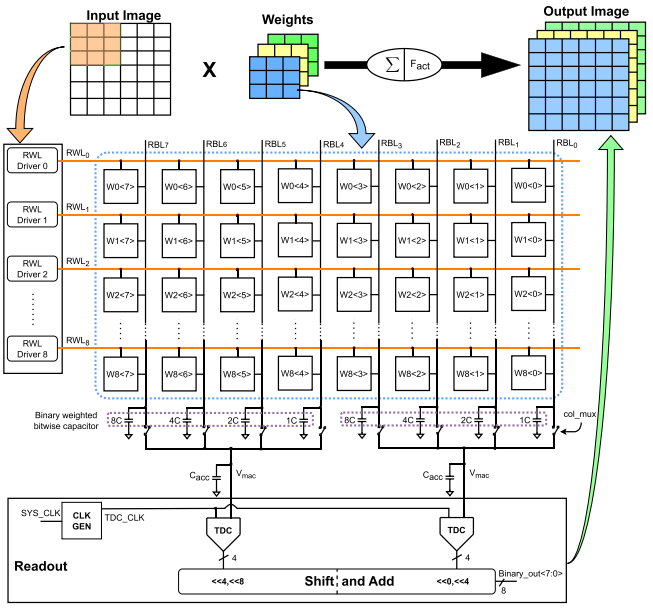


Fig. 4. Layer mapping of a convolution operation performed by applying input data through RWL drivers, multiplied with fixed weights in SRAM cells performing bitwise MAC operations along each bitline via binary-weighted capacitors, culminating in a multi-bit MAC output through charge sharing on the $C_{acc}$, where the resulting voltage $V_{mac}$ is digitized by the TDC.

This paper introduces a novel compute-in-memory (CiM) architecture that enhances the standard SRAM cache by enabling MAC operations within the memory structure. The design, detailed in Figure 2, integrates a conventional SRAM array with a binary-weighted capacitor array to perform analog MAC operations using charge-sharing for 8-bit inputs and 8-bit weights. A readout circuit formed by the proposed Time-to-Digital Converter (TDC) converts the analog MAC values to 2 sets of 4-bit digital values that are summed up to form an 8-bit MAC output.

### A. Proposed TDC-CiM Architecture

Figure 2 illustrates the architecture of the proposed TDC-CiM 8T SRAM macro, designed to facilitate MAC operations. This structure incorporates an 8T SRAM array, a binary-weighted capacitor array, and a readout circuit configured explicitly for MAC computations. The write-back of the MAC results is facilitated using the energy-recycling resonant write driver, enabling low-power write operations [33]. The architecture supports conventional read and write operations similar to traditional SRAM architectures with the help of row decoders for write wordlines ($WWL$) and read wordlines ($RWL$) and a peripheral control circuit to generate internal signals.

The 8T SRAM array consists of $256 \times 256$ bitcells to store and read using vertical bitlines and horizontal wordlines. During conventional read or write processes, the architecture is designed to activate either a single $RWL$ driver or a single $WWL$ driver at a time. However, nine $RWL$ drivers are simultaneously enabled to correspond with the $3 \times 3$ kernel of the input feature map (IFM) for executing MAC operations. A weight stationary dataflow scheme is adopted in this work and further detailed in Section III-E. During MAC operations, the column decoder selects the appropriate columns containing the stored weights. The main control unit generates a signal indicating the number of kernels (NoK), which is then decoded to activate the corresponding columns. The 8-bit filter weights ($W$) are stored as logic "1" or "0" onto the bitcells, and the 8-bit $IFMs$ are applied in 2 cycles as a series of analog pulses on the $RWLs$.

### B. Proposed Multibit Multiplication

In the proposed architecture, the 8-bit filter weights are loaded along the row direction in 2 sets of four adjacent 8T SRAM bitcells. The mapping of the filter weights inside the TDC-CiM array is shown in Figure 4. The 8-bit $IFMs$ are applied in two subsequent clock cycles as a series of 4-bit input pulses into the TDC-CiM array for MAC operation.

Figure 3(a) shows the transistor-level schematic of the 8T bitcell used for performing MAC operations. Figure 3(b)

illustrates bitwise multiplication using an 8T SRAM bitcell. Here, INPUT = "1" is represented by a unit pulse, whereas INPUT = "0" indicates no pulse. When the weight (Q-value) is set to "1," and the input is also "1," a current flow occurs, discharging the RBL line by $\Delta V$. Hence, it performs a 1-bit multiplication of stored weight and input for a single bitcell. The voltage drop $\Delta V$ due to discharge can be captured by quantizing the amount of charge ($Q$) lost from the capacitor ($C_{RBL}$). The remaining charge on the capacitor can be written as:

$$Q_{\text{remaining}} = Q_{\text{initial}} - Q_{\text{discharged}} \tag{1}$$

Since $Q = CV$, the remaining voltage ($V_{\text{rem}}$) on the capacitor ($C_{RBL}$) is:

$$V_{\text{rem}} \cdot C_{RBL} = (V_{\text{dd}} \cdot C_{RBL}) - (I_{\text{ds}} \cdot T_{\text{dis}}) \tag{2}$$

where, $I_{\text{ds}}$ is the current discharged for time period $T_{\text{dis}}$. Solving for $V_{\text{rem}}$:

$$V_{\text{rem}} = V_{\text{dd}} - \frac{I_{\text{ds}} \cdot T_{\text{dis}}}{C_{RBL}} \tag{3}$$

Thus, the voltage drop $\Delta V$ can be quantized as:

$$\Delta V = V_{\text{dd}} - V_{\text{rem}} = \frac{I_{\text{ds}} \cdot T_{\text{dis}}}{C_{RBL}} \tag{4}$$

Enabling multiple rows simultaneously lets us perform a series of these one-bit multiplications in parallel, leading to a collective discharge on the $RBL$ that represents the sum total of the individual multiplication operations, as shown in Figure 3(c). This process essentially accomplishes a bitwise MAC function across the RBL for the activated bitcells. Using 8T SRAM bitcells will overcome the limitation of reverse charging current in 6T/7T bitcells even when the bitwise multiplication yields a "0."

The total voltage drop on the $RBL$ due to $n$ parallel active rows can be expressed as:

$$\Delta V_{\text{total}} = \left(\sum_{i=0}^{n} X_i \cdot W_i\right) \cdot \left(\frac{I_{\text{ds}} \cdot T_{\text{dis}}}{C_{RBL}}\right) \tag{5}$$

where $X_i$ & $W_i$ represent the input features and kernel weights, respectively. The bitwise MAC results are sampled on two sets of capacitors: binary-weighted bitwise capacitors and the accumulation capacitor $C_{acc}$, as shown in Figure 4. The binary-weighted bitwise capacitors are discharged based on the number of rows activated. Once the bitwise multiplication is completed, the $col\_mux$ signal is turned "ON," enabling charge-sharing between the bitwise capacitors and the $C_{acc}$. The $C_{acc}$ output node is the $V_{mac}$ voltage corresponding to the analog MAC result of the inputs and weights.

Figure 4 illustrates the process of layer mapping in a CNN using an 8T SRAM cell array for performing MAC operations. The 8-bit $IFM$ is provided as input to the RWLs of the SRAM cells by the RWL drivers. A logical "1" in the input feature map generates a unit pulse on the RWL, while a logical "0" results in no pulse. Figure 5 shows the SPICE simulation results for this MAC operation.

The kernel weights, which are 8-bit stationary values, are stored horizontally adjacent to 8T SRAM bitcells within the same row. Each RBL is initially connected to a binary-weighted capacitor. During the multiplication phase, the initial precharged RBLs discharge incrementally, depending on the number of activated rows. The configuration depicted consists of nine wordlines corresponding to a $3\times3$ convolutional kernel typically used in CNN layers. Once the bitwise multiplication process is completed, we turn "ON" the $col\_mux$ signal that connects the binary-weighted capacitors to the $C_{acc}$. This initiates a charge-sharing mechanism that charges the $C_{acc}$ capacitor based on the combined charge from the binary-weighted capacitors, producing the final accumulated voltage $V_{mac}$, as shown in Figure 5.

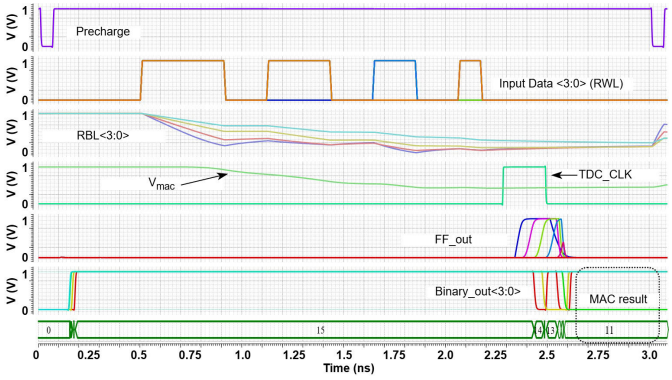


Fig. 5. The functionality simulation of performing a bitwise MAC operation on $RBLs$ and charge-sharing resulting in analog MAC voltage $V_{mac}$ which is digitized using the TDC.

The accumulated voltage $V_{mac}$ in $C_{acc}$ represents the MAC operation's result in analog form. This analog value $V_{mac}$ is fed into a TDC along with the $TDC\_CLK$ pulse. The TDC translates this $V_{mac}$ voltage into a 4-bit digital output ($Binary\_out < 3:0 >$), representing the partial MAC value, which in this case is "11."

In the proposed scheme, the 4-bit input sequence is multiplied with two sets of 4-bit weights, producing two separate 4-bit partial MAC values. These partial MAC results are combined over two clock cycles. In the first clock cycle, the lower 4 bits of the IFM are processed, and the corresponding partial outputs are left-shifted by 0 and 4 bits, respectively. In the second clock cycle, the remaining upper 4 bits of the IFM are processed in the same manner, with the partial outputs left-shifted by 4 and 8 bits, respectively. The shifted partial sums from both cycles are then added together to generate the final 8-bit MAC result. This final sum is stored in an output buffer, which is then written back into the SRAM array using a resonant write driver [34] for use in the next layer.

The simulated discharge rate of the RBLs and the $C_{acc}$ relative to the number of active rows is shown in Figure 6. As the number of rows activated increases, the discharge of both the RBLs and the accumulation capacitor also increases.

### C. Proposed TDC Architecture

Figure 7 illustrates the proposed 4-bit TDC. It comprises an array of pulse-shrinking delay elements alongside their corresponding D flip-flops ($DFFs$). The output ($Q$) of these $DFFs$ serve as inputs to the thermometer-to-binary encoder, resulting in the 4-bit binary output. The overall SPICE simulation of

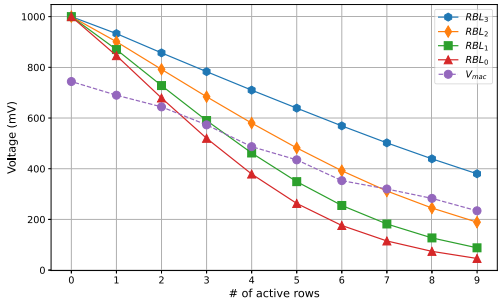


Fig. 6. Discharge rates of the read bitlines (RBLs) and the accumulation capacitor $C_{acc}$ demonstrate a linear decrease in voltage with increasing number of rows activated.

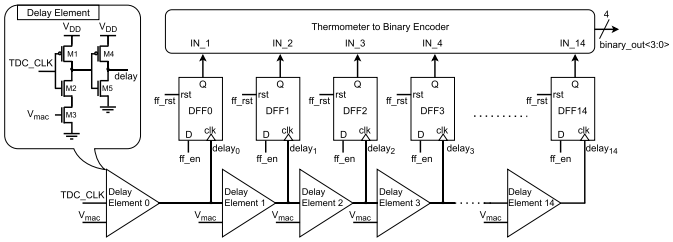


Fig. 7. The 4-bit TDC converter architecture comprises an array of pulse-shrinking voltage-controlled delay elements along with DFFs to generate a pulse count corresponding to the analog MAC value $V_{mac}$, converted into a digital output using a MUX-based thermometer-to-binary encoder.

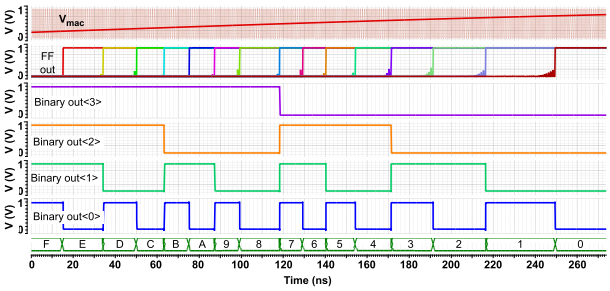


Fig. 8. The functionality simulation of the TDC shows a linear decrease of the digital output corresponding to the input voltage ramp signal $V_{mac}$, successfully capturing the full spectrum of 4-bit values.

the TDC circuit is shown in Figure 8. The TDC takes the analog output from the MAC operation, which is stored as a voltage ($V_{mac}$) in the $C_{acc}$, and turns it into a digital signal. The TDC also has an input pulse, $TDC\_CLK$, generated from the system clock using a buffer-based delay circuit.

The transistors $M1 - M5$, shown in Figure 7, form the pulse-shrinking delay element, which consists of the voltage-controlled buffer. The propagation of the rising edge of the input pulse ($TDC\_CLK$) is slowed down by the current-starving transistor $M3$ while the falling edge travels fast. The $V_{mac}$ result obtained from the charge sharing after bitwise multiplication is provided as the voltage control input to the TDC circuit. Each delay element will shrink the input pulse by $\Delta T$ depending upon the $V_{mac}$ result. As the pulse travels through the delay line, the width of the pulse shrinks in each element by $\Delta T$ until the pulse entirely disappears.

Each pulse-shrinking delay element is connected to the clock pin of a positive edge-triggered $DFF$. The $DFF$ latches a "1" using the propagated pulse until the pulse disappears, which results in the following $DFF$ to retain a "0." The

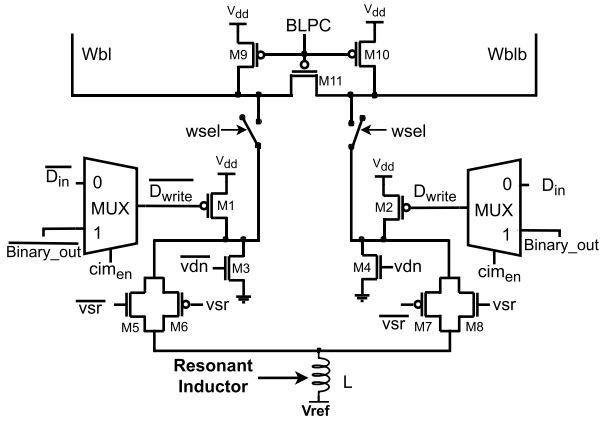


Fig. 9. The resonant write driver has 4 additional transistors (M5 to M8) and a shared inductor over the traditional write driver to enable energy recycling during every write operation.

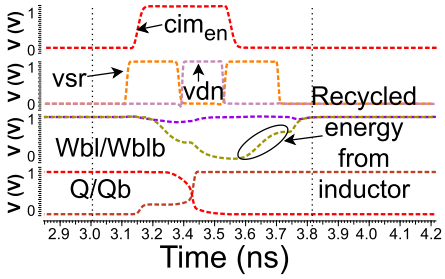


Fig. 10. Functionality validation of the resonant write driver showcasing the write operation of the TDC output ($binary\_out$) when $cim_{en}$ is enabled.

simulation depicted in Figure 8 captures the voltage required to set all the $DFFs$ to "1" by applying a voltage ramp signal.

The $Q$ from the $DFFs$ is subsequently provided as input to a MUX-based thermometer-to-binary encoder. This converter takes the string of "1s" and "0s" and converts them into a 4-bit binary number. This binary number is the digital version of the original analog voltage. This 4-bit digital output is provided as input to the shift&add module for the final 8-bit MAC output computations.

### D. Resonant Write Driver

The write path, shown in Figure 9, uses an energy-recycling resonant driver with supply boosting following prior work [19], [34]. A series inductor is placed in the discharge path of write bitlines $Wbl/Wblb$ and with the other end connected to a reference voltage $Vref$. Choosing $V_{\text{ref}} = V_{\text{DD}}/2$ maximizes the energy savings from the series inductor [34]. During a write operation, either $Wbl$ or $Wblb$ switches from logic "1" to logic "0," the charge that would otherwise be dissipated is transferred and stored at $V_{\text{ref}}$ node. During the subsequent precharge phase, this stored energy is returned from $V_{\text{ref}}$ to the bitlines, which reduces the energy drawn from the supply and yields zero net current over the write-precharge cycle.

The resonant write driver in Figure 9 uses transistors $M5 - M8$ to enable resonance by conditionally connecting the $Wbl/Wblb$ to the inductor controlled by the $vsr$ signal derived from the system clock. During the write of logic "1" case shown in Figure 10, the $vsr$ signal enables the discharge

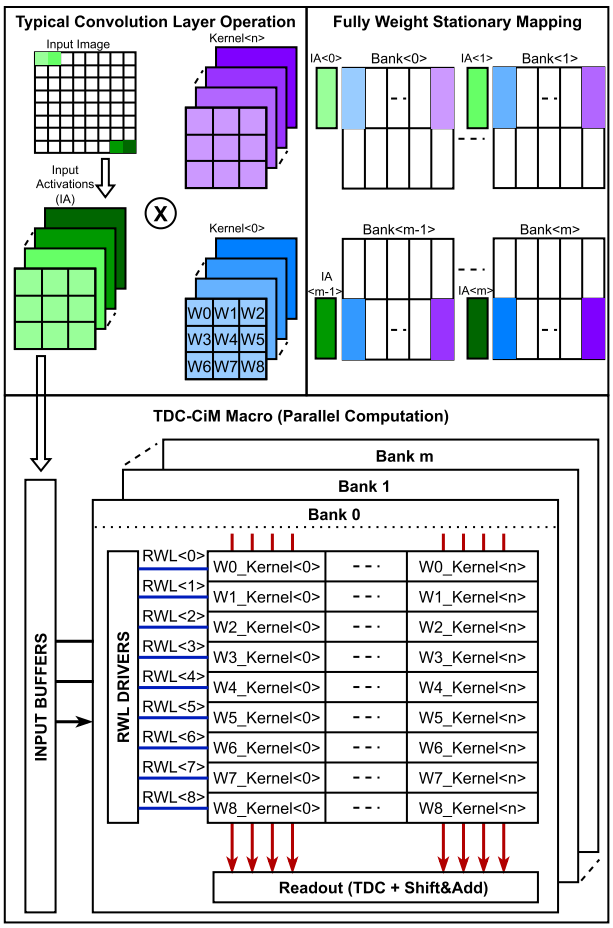


Fig. 11. The fully row-stationary weight mapping scheme of the proposed TDC-CiM Macro only requires sending the input activations each computation cycle, significantly reducing the latency for off-chip access of weights.

path of the *Wblb* signal for writing a data of "1." The *vdn* signal enables complete discharge of the bitline. During the subsequent precharge phase, *vsr* is asserted again so the energy stored in the inductor is returned to the bitlines. As a result, the precharge enable (*BLPC*) does not need to charge the bitline from 0V, reducing the overall energy consumption.

### *E. Weight Stationary Data Mapping Scheme*

Figure 11 illustrates the fully weight stationary data mapping scheme employed in the proposed TDC-CiM macro for CNN workloads. In this scheme, the 8-bit kernel weights for each CNN layer are first loaded and held stationary inside the SRAM banks, requiring only a single initialization cycle. This configuration minimizes unnecessary data movement by keeping the kernel weights local to each bank, while the input feature maps (IFMs) are supplied dynamically through the row wordline (RWL) drivers during computation. Towards data mapping, each TDC-CiM row stores 8-bit weights, and the input buffer employs two rows to store the 4-bit MSBs and 4-bit LSBs of the IFM from different channels. The weight stationary arrangement provides additional flexibility in how IFMs are mapped across the banks. When the number of kernels is large, the same IFM can be broadcast to all banks in parallel, allowing each bank to process a different kernel set with the same input data. Conversely, when the kernel count is small, multiple copies of the IFM can be distributed across different banks, enabling parallel computation on multiple IFM tiles within the same cycle. This choice between kernel parallelism and input parallelism helps balance throughput and utilization based on layer requirements. After each matrix-vector multiplication, the output feature maps are written back into the banks, ready for the next layer. This mapping strategy significantly reduces off-chip weight transfers and improves energy efficiency by exploiting local data reuse.

For example, in a typical LeNet-5 layer, the number of convolutional kernels is relatively small compared to large modern CNNs. In this case, the proposed weight stationary scheme can map the limited kernel set across only a portion of the available SRAM banks, freeing the remaining banks to process multiple IFM tiles in parallel during the same computation cycle. This allows the architecture to maintain high utilization and throughput even when the kernel count is low, by distributing different portions of the input data across separate banks for concurrent processing. For layers with larger kernel sets, the same mapping strategy can instead broadcast a single IFM to all banks to process a higher number of kernels simultaneously. This flexibility ensures that the TDC-CiM macro can adapt to diverse CNN layers, while minimizing redundant memory access and maximizing parallel computation. The control circuitry of the proposed weight stationary mapping scheme includes a programmable row decoder that enables selective activation of multiple RWLs across banks based on IFM positions, and a column decoder capable of indexing stationary kernel weights stored within the bitcells. The main controller coordinates these components to enable synchronized broadcast of IFM slices across banks and sequential MAC operations. Additionally, cycle-level control logic is required to manage the multi-cycle processing of 8-bit IFMs using two 4-bit partial accumulations per MAC operation.

Algorithm. 1 describes the workflow for selecting an optimal SRAM macro size to map a quantized CNN workload onto the proposed TDC-CiM architecture. The input to the algorithm is a pretrained neural network and a list of available SRAM macro configurations. The output is the resulting TDC-CiM hardware configuration tailored for the given model. First, the pretrained CNN is quantized to 8-bit integer precision, in Line. 3, to enable efficient low-bit in-memory computation. Next, Line. 4 extracts layer-wise structural information and the corresponding weights from the quantized model. Using this $LayerInfo$ & $Weights$, Line. 5 identifies a subset of suitable SRAM macro sizes based on layer dimensions, parameter count, and the varying level of parallelism.

For each chosen SRAM macro, the algorithm evaluates the power, latency, and energy performance in Lines. 6-7, considering the specific workload and hardware constraints. Once the evaluations are complete, the configuration yielding the lowest total energy consumption is selected by the filtering step in Line. 9. Finally, Line. 10 computes the required resonant inductor value for the selected SRAM size. This proposed methodology would result in the workload-specific

**Algorithm 1** Mapping Quantized CNN Workloads to Optimal TDC-CiM Architecture

**Input:** Pretrained Neural Network ($NN$), SRAM Macros ($SRAM_{list}$);
**Output:** rTD–CiM Architecture;
$QNN \leftarrow Quantize(NN, 8\text{-}bit)$; ▷ Quantize the NN to 8-bit INT precision
$LayerInfo, Weights \leftarrow Extract(QNN)$; ▷ Extract layer-wise information and weights
$SRAM_{size} \leftarrow IdentifySRAM(LayerInfo, Weights)$; ▷ Determine a set of SRAM's based on layer info and kernel counts
**for all** $SRAM$ in $SRAM_{size}$ **do** ▷ Loop through each SRAM Macro
  $Metrics_{list}[SRAM] \leftarrow Evaluate(LayerInfo, Weights, SRAM)$; ▷ Evaluate power, latency, and energy for each SRAM Macro chosen
**end for**
$BestSRAM \leftarrow SortEnergy(Metrics_{list})$; ▷ Determine lowest energy consuming AIG
$L_{res} \leftarrow CalculateInductor(BestSRAM)$; ▷ Compute resonant inductor size for chosen SRAM
**Output:** TDC-CiM Architecture $\leftarrow \{BestSRAM, L_{res}\}$; ▷ Resulting TDC-CiM architecture for the CNN model

TDC-CiM architecture that preserves quantization accuracy while maximizing energy efficiency and performance.

## IV. Experimental Results

### A. Experimental Setup

An 8 KB SRAM memory instance is designed and simulated using 28 nm TSMC technology. The SRAM memory array comprises 256 × 256 8T bitcells, implemented using Cadence Virtuoso, and simulations were performed utilizing the Cadence Spectre simulator at 0.5 GHz clock frequency. The inputs are driven using nine RWL drivers corresponding to a 3 × 3 input kernel of the $IFM$. The capacitor array, performing MAC computations, comprises binary-weighted bitwise capacitors and an $C_{acc}$. The bitwise LSB capacitor value is set to $1C$ = 4 fF, and the accumulation capacitor that stores the analog MAC output $V_{mac}$ is set to 32 fF. The readout circuitry comprises 64 TDC blocks, each connected to one capacitor array.

For system-level benchmarking, six neural network models were used, namely, LeNet-5 [35] (MNIST [36]), MobileNetV1 [37] and MobileNetV2 [38] (CIFAR-10 [39]), SqueezeNet [40] and ResNet-18 [41](ImageNet-1K [42]), and Tiny-YOLOv3 [43] (COCO [44]). All models were quantized to 8-bit precision using post-training quantization, without further fine-tuning. The proposed SRAM macro selection algorithm was applied to each network to identify the optimal SRAM size for minimizing energy consumption and inference latency. Energy and latency metrics were derived by mapping each layer to the SRAM configurations and accumulating per-layer estimates across the full network inference.

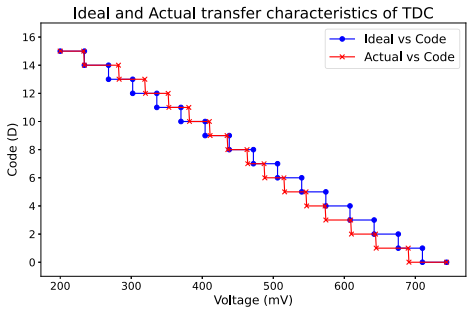


Fig. 12. Simulated transfer characteristics of TDC show a deviation from the linear ideal characteristic curve but track the analog MAC output voltage $V_{mac}$.

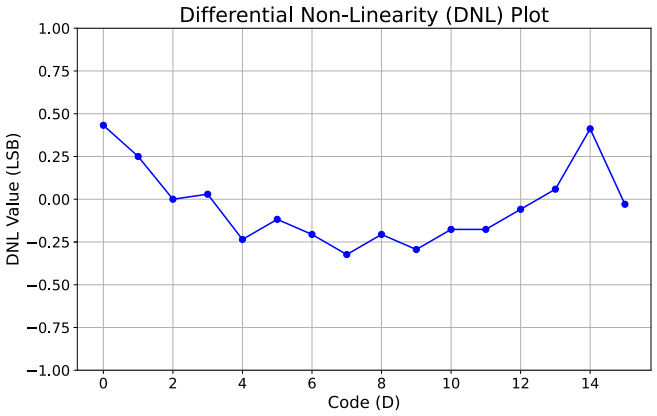


Fig. 13. The DNL characteristics plot showcases a tolerable deviation of -0.3/+0.4 LSBs.

### B. TDC Characterization and Comparison

Figure. 12 illustrates the transfer characteristics to analyze the linearity of the TDC. The input voltage applied to the TDC is plotted on the x-axis, with a baseline offset of 200 $mV$. The full voltage range of the proposed TDC circuit is from 200 $mV$ to 800 $mV$. The ideal performance of the TDC is depicted by the blue line, characterized by a consistent, monotonically decreasing function. In contrast, the simulated actual transfer characteristics of the proposed TDC circuit, shown in red, deviate from the linear function. This characteristic aligns with the linear discharge observed on the $C_{acc}$ as shown in Figure 6. The offset of the actual transfer curve can be corrected by performing the Differential Non-Linearity (DNL) and Integral Non-Linearity (INL) analysis, shown in Figure 13 and Figure 14, respectively.

Figure. 13 illustrates the DNL characteristics of the proposed TDC. The DNL is a measure of the deviation from the ideal step size between consecutive codes expressed in Least Significant Bits (LSB). A positive DNL value indicates that the actual step size between two successive codes is larger than one LSB, while a negative DNL value means the step size is smaller than one LSB. If the DNL value ever reaches -1 LSB, it would indicate a missing code, which is a serious non-linearity error in the TDC. However, the graph indicates that all of the DNL values are within ±0.5 LSB, which is considered to be tolerable bounds for TDC characterization. The plot showcases a maximum DNL value

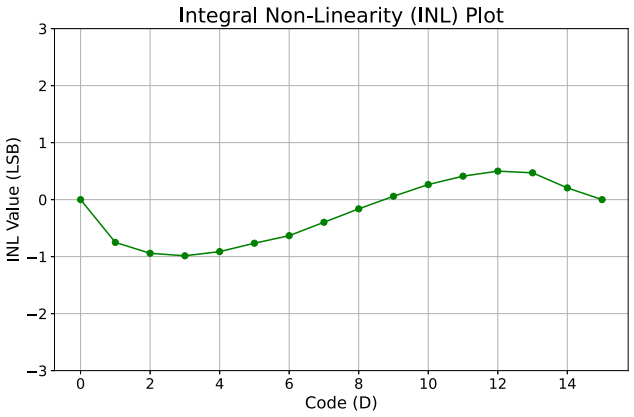


Fig. 14. The INL characteristics plot of TDC demonstrates a minimum deviation of -1 LSBs for digital code "2" and a maximum deviation of +0.5 LSBs for "12."

TABLE I

COMPARISON OF THE TDC ARCHITECTURE WITH PRIOR TD AND SAR ADCS SHOW LOWER POWER CONSUMPTION OF 45.6% COMPARED TO [8], AND 96% COMPARED TO [45]

| Reference | APCCAS'22 [8] | ESSCIRC'23 [45] | RFIC'18 [46] | JSSC'19 [9] | This Work |
|---|---|---|---|---|---|
| Architecture | TD ADC | SAR ADC | SAR ADC | TD ADC | **TDC** |
| Technology (nm) | 28 | 28 | 28 | 65 | **28** |
| Resolution (bits) | 9 | 6 | 8 | 8 | **4** |
| Fs (GS/s) | 0.5 | 1.4 | 8.8 | 1 | **1** |
| Supply (V) | 0.9 | 0.9 | 1.5 | 1 | **1** |
| SNDR (dB) | 54.69 | 67 | 38.4 | 45 | **19.45** |
| SFDR (dB) | 55.16 | NR | 48.9 | 60.3 | **22.4** |
| Power (mW) | 4.27 | 32 | 83.4 | 2.3 | **1.25** |
| FoM (fJ/conv.step) | 19.29 | 143.2 | 139.5 | 18.7 | **162.8** |

of + 0.4 LSB for digital code "0," and a minimum DNL value of -0.3 LSB for digital code "7."

Figure. 14 shows the INL plot for the proposed TDC. INL is a measure of the converter's linearity, indicating the maximum deviation from the ideal function mapping of input to output over the full range of the converter. The INL plot exhibits an initial positive deviation, signifying that the early output codes from the TDC are larger than the expected values for an ideally linear system. As the input value increases, the INL plot trends downward, eventually falling below the zero level, which indicates that the output codes from the TDC are incrementally lower than what would be predicted by a linear model. The plot shows a maximum INL value of + 0.5 LSB for digital code "12," and a minimum INL value of -1 LSB for digital code "2."

Table. I compares various Analog-to-Digital Converter (ADC) architectures across several design references. The resolution of the TDC used in this work is 4 bits at a sampling rate of 1GS/s. The voltage supply is 1V for the TDC. The TDC exhibits an SNDR of 19.45 dB and an SFDR of 22.4 dB. The TDC utilizes $V_{mac}$ as its input, eliminating the need for the voltage-to-time converters typically required in conventional TDC architectures. This approach reduces the overall power consumption of the TDC framework. The proposed TDC achieves 71% lower power consumption than [8] and 45.6% lower power consumption than [9], which are time-domain analog-to-digital converters ($TD\ ADC$). Additionally, the proposed TDC demonstrates 98% lower power consumption than [46] and 96% lower power consumption than [45], which are $SAR\ ADCs$, typically employed in conventional CIM architectures. The TDC achieves a Walden Figure of Merit ($FoM$) of 162.8 fJ/conversion step, which is 12% higher than [45] and 14% higher than [46]. This FoM, which is directly correlated to the bit resolution, can be reduced by increasing the number of output bits of the TDC.

### C. Process, Voltage, and Temperature Variation Analysis

To evaluate the robustness of the proposed TDC design, we performed extensive Monte Carlo simulations considering PVT variations. We performed a Monte Carlo analysis of the proposed TDC under supply voltage variations, using 2000 samples for each voltage corner with a $\pm 10\%$ variation from the nominal voltage. The results, shown in Fig. 15, illustrate the impact of supply voltage on power and delay characteristics. For $V_{DD}$ = 0.9 V, the mean power consumption is 334.8 $\mu$W with a standard deviation of 1.72 $\mu$W, and the mean delay is 247 ps with a standard deviation of 2.68 ps. Increasing the supply voltage to $V_{DD}$ = 1 V results in a mean power of 432 $\mu$W ($\sigma$ = 2.29 $\mu$W) and a mean delay of 285 ps ($\sigma$ = 2.09 ps). At $V_{DD}$ = 1.1 V, the mean power further increases to 512.37 $\mu$W with a standard deviation of 2.93 $\mu$W, while the mean delay reduces to 309.56 ps with a standard deviation of 1.79 ps. The mean of the delay variation corresponds to the pulse width of a single delay stage, where a larger pulse width indicates a lower overall TDC delay. These results confirm that higher supply voltages increase power consumption but reduce delay, demonstrating the expected trade-off for voltage scaling in the TDC design.

We also performed a Monte Carlo analysis of the proposed TDC under temperature variations, using 2000 samples for each temperature corner. The temperature was varied across three representative operating points: 0 °C, 27 °C, and 100 °C, to assess the impact of thermal conditions on power and delay performance. As shown in Fig. 16, the mean power consumption at 0 °C is 430.6 $\mu$W with a standard deviation of 3.4 $\mu$W, while the mean delay is 287.5 ps with a standard deviation of 2.05 ps. At the nominal temperature of 27 °C, the mean power is 432 $\mu$W with $\sigma$ = 2.29 $\mu$W and the mean delay is 285 ps with $\sigma$ = 2.09 ps. For high temperature operation at 100 °C, the mean power increases to 485.44 $\mu$W with a standard deviation of 2.13 $\mu$W, and the mean delay decreases to 279.82 ps with a standard deviation of 2.2 ps. Similar to the voltage variation results, the mean of the delay variation represents the pulse width of a single delay stage, where a higher pulse width corresponds to a lower overall TDC delay. These results indicate that higher temperatures slightly increase power consumption and reduce delay due to enhanced carrier mobility at elevated temperatures.

### D. Benchmarking With Neural Network Models

We have evaluated the proposed TDC-CiM architecture on six different neural network models, ranging from small networks with 60K parameters to large models with over 11M parameters. Specifically, we used the LeNet-5 [35] model trained on the MNIST [36] dataset, MobileNetV1 [37] and MobileNetV2 [38] trained on the CIFAR-10 [39] dataset,

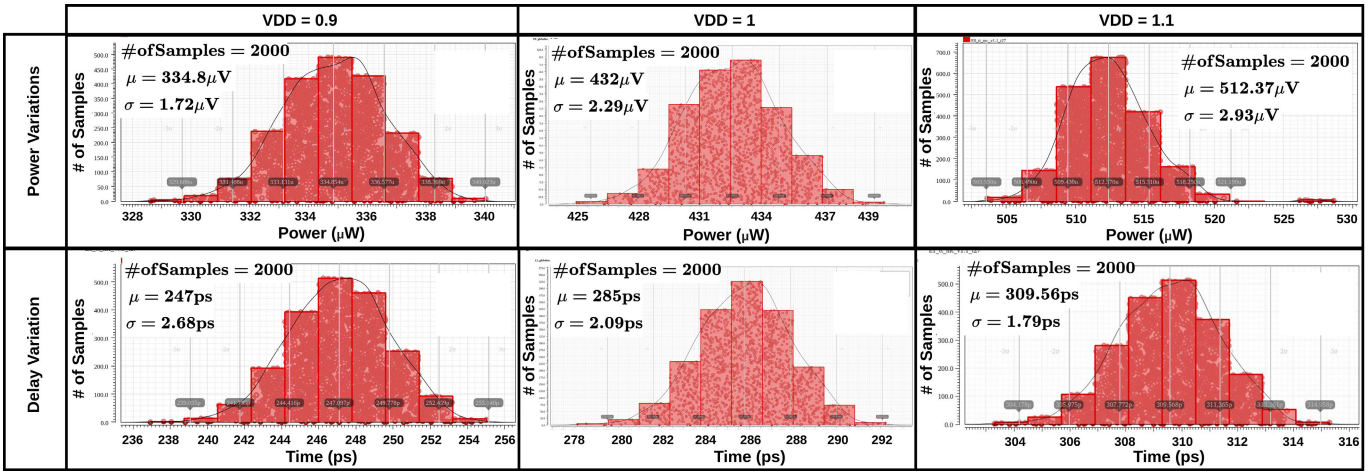


Fig. 15. Monte Carlo analysis of the proposed TDC under supply voltage variations. The top row shows the power variation for $V_{DD}$ = 0.9 V, 1 V, and 1.1 V, respectively. The bottom row shows the corresponding delay variation distributions for each voltage corner. A total of 2000 samples were simulated for each case using $3\sigma$ process variations and $\pm 10\%$ supply voltage variation, demonstrating that increasing the supply voltage increases the mean power consumption while reducing the mean delay, consistent with the expected behavior of voltage scaling in delay stages.

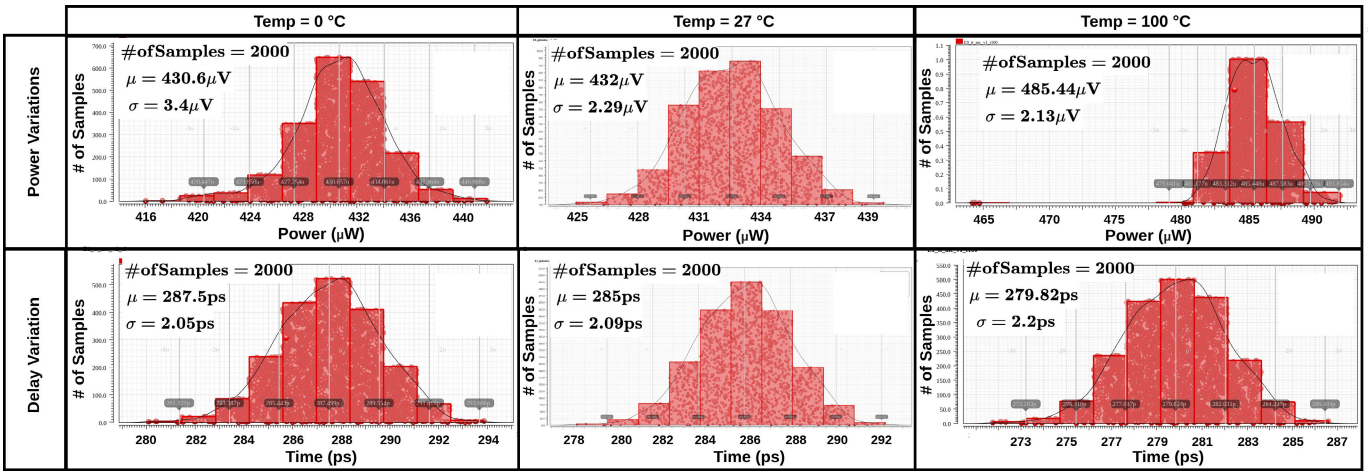


Fig. 16. Monte Carlo analysis of the proposed TDC under temperature variations. The top row shows power variation for 0 °C, 27 °C, and 100 °C, while the bottom row shows the corresponding delay variation. Each corner uses 2000 samples with $3\sigma$ process variation, showing that higher temperatures slightly increase power and reduce delay while maintaining stable performance across the full operating range.

TABLE II

BENCHMARKING ANALYSIS OF THE NEURAL NETWORK MODELS REVEALS A MINIMAL ACCURACY DROP, CONSIDERING BOTH FULL PRECISION AND INT8 DATA TYPES, AS WELL AS THE OPTIMAL SRAM SIZES FOR ACHIEVING THE OPTIMAL ENERGY EFFICIENCY

| Model | Dataset | Parameters | Full Precision Accuracy | INT8 Accuracy (Software) | INT8 Accuracy (TDC-CiM) | SRAM Size |
|---|---|---|---|---|---|---|
| **LeNet-5 [35]** | MNIST [36] | 61.47K | 98.76% | 98.7% | 98.7% | 24 KB |
| **MobileNetV1 [37]** | CIFAR-10 [39] | 3.19M | 92.45% | 89.73% | 89.6% | 64 KB |
| **MobileNetV2 [38]** | CIFAR-10 [39] | 2.35M | 96.43% | 96.48% | 96.1% | 64 KB |
| **SqueezeNet [40]** | ImageNet-1K [42] | 1.24M | 58.18% | 56.50% | 55.80% | 128 KB |
| **ResNet-18 [41]** | ImageNet-1K [42] | 11.68M | 67.61% | 67.14% | 67.1% | 256 KB |
| **Tiny-YOLOv3 [43]** | COCO [44] | 8.85M | 33% (mAP) | 31% (mAP) | 31% (mAP) | 256 KB |

SqueezeNet [40] and ResNet-18 [41] trained on the ImageNet-1K [42] dataset, and Tiny-YOLOv3 [43] trained on the COCO [44] dataset.

Table II reports the full-precision accuracy alongside the quantized INT8 accuracy, demonstrating that quantization introduces minimal accuracy loss. The pretrained models were quantized without any further fine-tuning. Depending on the model size, varying SRAM sizes enable more energy-efficient implementations. Figure 17 shows the power, latency, and energy consumption comparison for these neural network benchmarks. A detailed power, latency, and energy analysis was performed across nine different SRAM macro sizes and varying numbers of banks.

Figure 17(a) compares the overall power consumption of each neural network benchmark when executed on the proposed TDC-CiM architecture across varying SRAM macro sizes. The total power consumption remains approximately constant for a given workload, regardless of the SRAM size. This behavior can be explained by considering the balance between parallelism and cycle count: for instance, a 16 KB SRAM macro provides twice the storage compared to an 8 KB macro, allowing twice as many MAC operations to be computed in parallel per clock cycle. However, the smaller 8 KB configuration requires twice as many cycles to complete

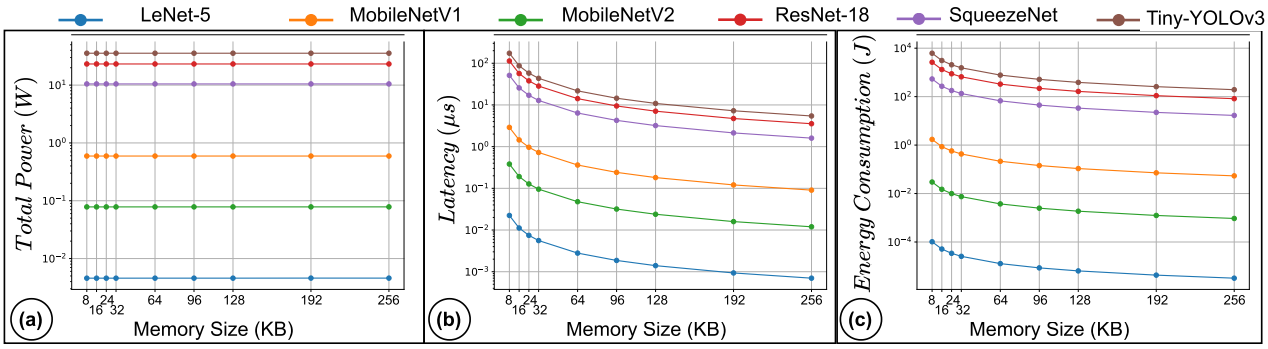


Fig. 17. Evaluation of CNN models on SRAM macros ranging from 8 KB to 256 KB. (a) Total power consumption remains mostly constant across memory sizes, with deeper models consuming more power. (b) Inference latency decreases with larger SRAM sizes, with greater reduction observed in deeper models due to fewer memory accesses. (c) Energy consumption decreases with memory size, indicating improved efficiency from reduced off-chip data movement.

the same total number of MAC operations for a fixed model size. As a result, the total dynamic switching activity remains nearly unchanged, and the static power overhead is effectively spread across the entire computation time, leading to negligible variation in overall power consumption across different SRAM macro sizes.

Figure 17(b) depicts the inference latency for all neural network benchmarks across different SRAM macro sizes. On average, there is a latency reduction of 70% when increasing the macro size from 8 KB to 24 KB, highlighting the benefit of greater local storage in enabling more MAC operations per cycle and reducing the number of sequential compute cycles. A further increase from 16 KB to 64 KB achieves an additional average latency reduction of around 60%, demonstrating the improved parallel data access and reduced need for repeated weight fetching. Similarly, scaling the macro from 128 KB to 256 KB yields an average latency improvement of about 45%, which shows diminishing returns at larger macro sizes due to saturation of available parallelism relative to the workload size. This clear trend confirms that larger SRAM macros in the TDC-CiM architecture effectively exploit intra-cycle parallelism and data reuse, minimizing memory bottlenecks and accelerating inference performance for a wide range of network sizes.

Figure 17(c) illustrates the average energy consumption per inference for all benchmark models across different SRAM macro sizes. On average, expanding the macro from 8 KB to 32 KB reduces the total energy by 67%, primarily by minimizing the number of sequential memory access cycles required for weight and activation fetches. Increasing the macro size further from 32 KB to 96 KB yields an additional average energy reduction of 73.6%, highlighting the advantage of higher data reuse and fewer off-chip transactions. Extending the SRAM from 96 KB to 256 KB results in a further 62% drop in energy, showing that larger macros maintain significant efficiency gains even at high SRAM memory size. These results confirm that the proposed TDC-CiM architecture can achieve substantial energy savings by leveraging large SRAM arrays to maximize in-memory operations and reduce redundant data movement between compute and storage units.

Table II summarizes the benchmarked neural network models, including dataset, parameter count, accuracy before and after INT8 quantization, and the selected SRAM macro size used for optimal inference performance. On average, the quantization to INT8 results in an average accuracy reduction of 0.75% across the five CNN models evaluated. The optimal SRAM size for each model balances memory capacity with the required level of parallel computation. Smaller models such as LeNet-5 [35] can achieve maximum efficiency with relatively small 24 KB SRAM blocks since their total parameter count is low. In contrast, larger models like ResNet-18 [41] and Tiny-YOLOv3 [43] require significantly more on-chip storage to exploit high levels of parallel MAC operations and minimize repeated weight fetching, justifying the use of large 256 KB SRAM configurations. Although SqueezeNet [40] has a relatively low parameter count compared to other ImageNet-class models, its architectural design involves a large number of operations per parameter due to its squeeze-and-expand modules. As a result, it demands a higher 128 KB SRAM size than its raw parameter count might suggest, to sustain high parallelism and reduce the overall cycle count. Table II also demonstrates that selecting the appropriate SRAM size for each network ensures low quantization-induced accuracy drop while achieving substantial energy and latency improvements through compute-in-memory parallelism.

TABLE III

COMPARISON OF THE PROPOSED TDC-CIM ARCHITECTURE WITH PRIOR CIM WORKS SHOW 59.7% HIGHER ENERGY EFFICIENCY COMPARED TO [47] AND ACHIEVES 6.25× HIGHER THROUGHPUT THAN [30]

| | This work | JSSC'23 [47] | JSSC'24 [30] | TCAS-I'24 [48] | ISSCC'22 [49] |
|---|---|---|---|---|---|
| **Technology (nm)** | **28** | 22 | 28 | 28 | 28 |
| **Cell Type** | **8T** | 9T | 8T | 8T1C | 6T |
| **Array Size** | **16KB** | 128KB | 16KB | 64KB | 1MB |
| **Precision (input/weight)** | **8/8** | 8/8 | 8/8 | 8/8 | 8/8 |
| **Supply Voltage (V)** | **1** | 1.1 | 0.9 | 0.8 | 0.9 |
| **Frequency (GHz)** | **0.5** | 0.24 | 0.2 | 0.25 | - |
| **Throughput (GOPS)** | **320** | 1000 | 51.2 | 256 | 1241 |
| **Energy Efficiency (TOPS/W)** | **38.46** | 15.5 | 22.2 | 71.17 | 37.01 |
| **Compute Density (TOPS/$mm^2$)** | **2.1** | 4 | 1.6 | 1.29 | - |

### E. MAC Comparison With Previous Works

Table III compares the proposed TDC-CiM architecture with existing CiM architectures capable of performing MAC operations. The proposed architecture can perform MAC operations for 8-bit inputs and 8-bit weights using 8T bitcells at 0.5 $GHz$ clock frequency. Table III reports our numbers from

transistor-level, post-layout SPICE simulations of 512×256 8T TDC-CIM macro that includes wordline and bitline drivers, local control, and the 4-bit TDC. All simulations use a 28 nm CMOS process at TT and 25 °C with $VDD$ = 1.0 V. The TDC input range is 200 to 800 mV based on the MAC discharge range. INT8 inference is implemented as two 4-bit MAC cycles with shift and add accumulation. Throughput is computed from simulated cycle counts and clock frequency using the configured parallelism for a 16 KB SRAM macro. Energy efficiency (TOPS/W) is the throughput divided by total macro power, where the power includes dynamic and leakage from the CiM array, the wordline and bitline drivers, the TDC readout, and local control.

The TDC-CiM achieves a throughput of 320 $GOPS$ with an energy efficiency of 38.46 $TOPS/W$. In [47], a 9T bipolar cell with differential inputs is used to perform XOR operations, which leads to higher leakage power due to the continuous current paths. In contrast, the 8T cell employed in this work disconnects the capacitors when idle, effectively reducing leakage. As a result, the proposed architecture achieves twice the operating frequency and a 59.7% improvement in energy efficiency compared to [47]. In [30], DAC/ADC-based in-memory operations contribute to high energy consumption, while more than 50% of the total area is occupied by the data converters used for CiM computations in [48]. In contrast, the proposed TDC-CiM architecture eliminates the need for an ADC, significantly reducing energy overhead and area usage. As a result, the TDC-CiM achieves a 6.25× increase in throughput, 1.31× higher area efficiency and a 42.2% improvement in energy efficiency compared to [30], while also delivering 2× higher frequency and 20% higher throughput, and 1.62× higher area efficiency than [48].

## V. Conclusion

This paper presents a TDC-CiM architecture designed for low-power in-memory computation. The architecture performs bitwise multiplications using 8T SRAM bitcells and leverages a binary-weighted capacitor array to execute MAC operations directly within the memory. A custom time-to-digital converter (TDC) digitizes the analog MAC results, eliminating the need for energy- and area-intensive ADCs commonly used in conventional CiM designs. The proposed TDC achieves a sampling frequency of 1 GS/s with 1.25 mW power consumption, an SNDR of 19.45 dB, and a Walden FoM of 162.8 fJ/conversion-step.

To support scalable deployment across CNN workloads, an automated SRAM macro selection algorithm is proposed using a fully weight stationary mapping scheme. Benchmarking across six neural networks with SRAM sizes ranging from 8 KB to 256 KB shows that increasing the SRAM size results in significant energy savings: up to 67% from 8 KB to 32 KB, an additional 73.6% from 32 KB to 96 KB, and a further 62% from 96 KB to 256 KB. These improvements are attributed to reduced memory access cycles, increased in-memory data reuse, and improved MAC parallelism.

The proposed TDC-CiM architecture demonstrates a throughput of 320 GOPS with an energy efficiency of 38.46 TOPS/W, confirming its potential for energy-constrained edge AI applications requiring parallel and low-latency MAC operations.

## References


[1] G. Singh, A. Wagle, S. Khatri, and S. Vrudhula, "CIDAN-XE: Computing in DRAM with artificial neurons," *Frontiers Electron.*, vol. 3, Feb. 2022, Art. no. 834146.

[2] F. Zhang, S. Angizi, J. Sun, W. Zhang, and D. Fan, "Aligner-D: Leveraging in-DRAM computing to accelerate DNA short read alignment," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 332–343, Mar. 2023.

[3] G. Singh and S. Vrudhula, "A scalable and energy-efficient processing-in-memory architecture for gen-AI," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 15, no. 2, pp. 285–298, Jun. 2025.

[4] I. Choi et al., "An SRAM-based hybrid computation-in-memory macro using current-reused differential CCO," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 12, no. 2, pp. 536–546, Jun. 2022.

[5] X. Li et al., "Full-array Boolean logic CIM macro with self-recycling 10T-SRAM cell for AES systems," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 8, pp. 2214–2224, Aug. 2025.

[6] S. Kim, S. Kim, S. Um, S. Kim, K. Kim, and H.-J. Yoo, "Neuro-CIM: ADC-less neuromorphic computing-in-memory processor with operation gating/stopping and digital–analog networks," *IEEE J. Solid-State Circuits*, vol. 58, no. 10, pp. 2931–2945, Oct. 2023.

[7] U. Saxena, I. Chakraborty, and K. Roy, "Towards ADC-less compute-in-memory accelerators for energy efficient deep learning," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2022, pp. 624–627.

[8] Y. Zhao, F. Ye, and J. Ren, "A 500-MS/s 9-bit time-domain ADC using a nonbinary successive approximation TDC," in *Proc. IEEE Asia Pacific Conf. Circuits Syst. (APCCAS)*, Nov. 2022, pp. 209–212, doi: 10.1109/APCCAS55924.2022.10090395.

[9] K. Ohhata, "A 2.3-mW, 1-GHz, 8-bit fully time-based two-step ADC using a high-linearity dynamic VTC," *IEEE J. Solid-State Circuits*, vol. 54, no. 7, pp. 2038–2048, Jul. 2019.

[10] D. Challagundla, I. Bezzam, and R. Islam, "A resonant time-domain compute-in-memory (rTD-CiM) ADC-less architecture for MAC operations," in *Proc. Great Lakes Symp. VLSI*, Jun. 2024, pp. 268–271, doi: 10.1145/3649476.3658773.

[11] J. Lou, F. Freye, C. Lanius, and T. Gemmeke, "Scalable time-domain compute-in-memory BNN engine with 2.06 POPS/W energy efficiency for edge-AI devices," in *Proc. Great Lakes Symp. VLSI*, Mar. 2023, pp. 665–670, doi: 10.1145/3583781.3590220.

[12] S. Angizi, A. Roohi, M. Taheri, and D. Fan, "Processing-in-Memory acceleration of MAC-based applications using residue number system: A comparative study," in *Proc. Great Lakes Symp. VLSI*, Jun. 2021, pp. 265–270, doi: 10.1145/3453688.3461529.

[13] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.

[14] K. Xiao et al., "A 28 nm 32Kb SRAM computing-in-memory macro with hierarchical capacity attenuator and input sparsity-optimized ADC for 4b MAC operation," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 6, pp. 1816–1820, Jun. 2023.

[15] D. Kushwaha et al., "A 65 nm compute-in-memory 7T SRAM macro supporting 4-bit multiply and accumulate operation by employing charge sharing," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Mar. 2022, pp. 1556–1560.

[16] F. Wei, X. Cui, S. Zhang, and X. Zhang, "An 11T SRAM cell for the dual-direction in-array Logic/CAM operations," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 4, pp. 2329–2333, Apr. 2024.

[17] S. Zhang, X. Cui, F. Wei, and X. Cui, "An area-efficient in-memory implementation method of arbitrary Boolean function based on SRAM array," *IEEE Trans. Comput.*, vol. 72, no. 12, pp. 3416–3430, Dec. 2023.

[18] D. Challagundla, I. Bezzam, and R. Islam, "Design automation of series resonance clocking in 14-nm FinFETs," *Circuits, Syst., Signal Process.*, vol. 42, no. 12, pp. 7549–7579, Dec. 2023.

[19] R. Islam, D. Challagundla, and I. Bezzam, "*System and methods of reducing wideband series resonant clock skew*," U.S. Patent 18 627 479, Oct. 10, 2024.

[20] R. Islam, "Low-power resonant clocking using soft error robust energy recovery flip-flops," *J. Electron. Test.*, vol. 34, no. 4, pp. 471–485, Aug. 2018.

[21] J. Rosenfeld and E. G. Friedman, "Design methodology for global resonant H-tree clock distribution networks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 15, no. 2, pp. 135–148, Feb. 2007.

[22] H. You, W. Li, D. Shang, Y. Zhou, and S. Qiao, "A 1–8b reconfigurable digital SRAM compute-in-memory macro for processing neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 4, pp. 1602–1614, Apr. 2024.
[23] C. Eckert et al., "Neural cache: Bit-serial in-cache acceleration of deep neural networks," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2018, vol. 39, no. 3, pp. 383–396.
[24] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 217–230, Jan. 2019.
[25] J. Song et al., "TD-SRAM: Time-domain-based in-memory computing macro for binary neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 8, pp. 3377–3387, Aug. 2021.
[26] S. Cheon, K. Lee, and J. Park, "A 2941-TOPS/W charge-domain 10T SRAM compute-in-memory for ternary neural network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 2085–2097, May 2023.
[27] J. Song, X. Tang, X. Qiao, Y. Wang, R. Wang, and R. Huang, "A 28 nm 16 kb bit-scalable charge-domain transpose 6T SRAM in-memory computing macro," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 1835–1845, May 2023.
[28] Y. Chen et al., "SAMBA: Single-ADC multi-bit accumulation compute-in-memory using nonlinearity- compensated fully parallel analog adder tree," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 7, pp. 2762–2773, Jul. 2023.
[29] Z. Chen et al., "PICO-RAM: A PVT-insensitive analog compute-in-memory SRAM macro with in situ multi-bit charge computing and 6T thin-cell-compatible layout," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 308–320, Jan. 2025.
[30] K. Lee, J. Kim, and J. Park, "A 28-nm 50.1-TOPS/W P-8T SRAM compute-in-memory macro design with BL charge-sharing-based in-SRAM DAC/ADC operations," *IEEE J. Solid-State Circuits*, vol. 59, no. 6, pp. 1926–1937, Jun. 2024.
[31] P. Chen, S.-I. Liu, and J. Wu, "A low power high accuracy CMOS time-to-digital converter," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, vol. 1, Jun. 2002, pp. 281–284, doi: 10.1109/ISCAS.1997.608704.
[32] E. Raisanen-Ruotsalainen, T. Rahkonen, and J. Kostamovaara, "A low-power CMOS time-to-digital converter," *IEEE J. Solid-State Circuits*, vol. 30, no. 9, pp. 984–990, Sep. 1995, doi: 10.1109/4.406397.
[33] R. Islam, B. Saha, and I. Bezzam, "Resonant energy recycling SRAM architecture," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 4, pp. 1383–1387, Apr. 2021.
[34] D. Challagundla, I. Bezzam, and R. Islam, "ArXrCiM: Architectural exploration of application-specific resonant SRAM compute-in-memory," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 1, pp. 179–192, Jan. 2025.
[35] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[36] Y. LeCun, C. Cortes, and C. J. Burges. (1998). *The MNIST Database of Handwritten Digits*. [Online]. Available: https://yann.lecun.com/exdb/mnist
[37] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
[38] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.
[39] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Tech. Rep. 1, 2009.
[40] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5 MB model size," 2016, *arXiv:1602.07360*.
[41] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.
[42] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2009, pp. 248–255.
[43] J. Redmon and A. Farhadi. (2018). *YOLOv3: An Incremental Improvement*. [Online]. Available: https://pjreddie.com/darknet/yolo/
[44] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.
[45] L. M. Santana, E. Martens, J. Lagos, P. Wambacq, and J. Craninckx, "A 70MHz bandwidth time-interleaved noise-shaping SAR assisted delta sigma ADC with digital cross-coupling in 28nm CMOS," in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2023, pp. 389–392, doi: 10.1109/ESSCIRC59616.2023.10268759.
[46] X. S. Wang et al., "An 8.8-GS/s 8b time-interleaved SAR ADC with 50-dB SFDR using complementary dual-loop-assisted buffers in 28nm CMOS," in *Proc. IEEE Radio Freq. Integr. Circuits Symp. (RFIC)*, Jun. 2018, pp. 88–91, doi: 10.1109/RFIC.2018.8429007.
[47] H. Wang, R. Liu, R. Dorrance, D. Dasalukunte, D. Lake, and B. Carlton, "A charge domain SRAM compute-in-memory macro with C-2C ladder-based 8-bit MAC unit in 22-nm FinFET process for edge inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1037–1050, Apr. 2023.
[48] X. Qiao et al., "A 16.38TOPS and 4.55POPS/W SRAM computing-in-memory macro for signed operands computation and batch normalization implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 4, pp. 1706–1718, Apr. 2024.
[49] P.-C. Wu et al., "A 28 nm 1Mb time-domain computing-in-memory 6T-SRAM macro with a 6.6ns latency, 1241GOPS and 37.01TOPS/W for 8b-MAC operations for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, Feb. 2022, pp. 1–3, doi: 10.1109/ISSCC42614.2022.9731681.

**Dhandeep Challagundla** (Graduate Student Member, IEEE) received the M.S. degree from the University of Maryland, Baltimore County (UMBC), Baltimore, MD, USA, where he is currently pursuing the Ph.D. degree with the Computer Science and Electrical Engineering Department. His research interests include energy-efficient computing, compute-in-memories, SRAM design, low-power circuit design, mixed-signal IC design, and EDA tools.

**Ignatius Bezzam** (Member, IEEE) received the Bachelor of Technology degree from IIT Madras, India, in 1983, and the Ph.D. degree in electrical engineering from Santa Clara University, in 2015. He holds several key patents in analog mixed signal integrated circuit (IC) design with publications in top international conferences, including ISSCC, ESSCIRC, and TCAS. He has owned 30 first silicon successes with global teams, with 33 years of next generation chip design experience in Silicon Valley, Europe, and Asia.

**Riadul Islam** (Senior Member, IEEE) designed the first current-pulsed flip-flop/register that resulted in the first-ever one-to-many current-mode clock distribution networks for high-performance microprocessors during his Ph.D. dissertation work at UCSC. From 2017 to 2019, he was an Assistant Professor with the University of Michigan, Dearborn, MI, USA. He is currently an Assistant Professor with the Department of Computer Science and Electrical Engineering, University of Maryland, Baltimore County. He holds two U.S. patents and several IEEE/ACM/MDPI/Springer Nature journal and conference publications. His current research interests include digital, analog, and mixed-signal CMOS ICs/SOCs for a variety of applications; verification and testing techniques for analog, digital, and mixed-signal ICs; hardware security; CAN network; CAD tools for design and analysis of microprocessors and FPGAs; automobile electronics; and biochips. He is a member of ACM, the IEEE Circuits and Systems (CAS) Society, the VLSI Systems and Applications Technical Committee (VSA-TC) of the IEEE-CAS, and the IEEE Solid-State Circuits (SSC) Society. He was a Technical Program Committee (TPC) Member of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD 2022), the ACM Great Lakes Symposium on VLSI (GLSVLSI 2020, GLSVLSI 2021, and GLSVLSI 2022), the 57th IEEE/ACM Design Automation Conference (DAC) 2020 LBR Session, the IEEE Computer Society Annual Symposium on VLSI (ISVLSI) 2021, and the IEEE International Conference on Consumer Electronics (ICCE) 2021. He was a recipient of the 2021 NSF ERI Award, the 2021 Maryland Industrial Partnerships (MIPS) Award, and the 2021 Maryland Innovation Initiative (MII) Award. He is an Associate Editor of *Circuits, Systems, and Signal Processing* (CSSP, Springer).